

AA-09

Operations, Free-Tier Budget and Runbook

The zero-rupee ledger for three more front ends, alert thresholds, and eleven runbooks

30 August 2026 · Rev 1.0 — baseline · AA-09 of thirteen

MONTHLY BILL	RECURRING INFRA	BUDGET LINES
₹67	Rs 0	16
CRON WORKFLOWS	RUNBOOKS	PRICED GAPS
11	11	5

Two documents live in here. The first is the ledger a founder opens when asked what this costs to run: sixteen free-tier lines, the number at which each one is breached, the pre-agreed degradation, and five things that genuinely cost money with their real prices. The second is the pack somebody opens at 02:00 when a shipment is stuck, cash is missing or a staff account has been phished: eleven runbooks written as steps, not as reasoning. Read AA-09 section 9.2 before you believe the cost line, because the polling arithmetic behind it is arithmetic and not measurement.

9.1 WHAT RUNNING THIS COSTS

Rs 0 per month recurring, ₹67 amortised, and the invariant that keeps it there

The recurring infrastructure bill for this programme is zero rupees per month. The total bill is the domain registration amortised over twelve months: ₹67 per month, unchanged from the pre-portal baseline. Three new front ends, one new package registry and three new hostnames add ₹0. That is not a happy accident; it is a design constraint that was applied before any technology was chosen, and several otherwise better choices lost to it.

Invariant I-14 is the rule that holds it. It forbids two specific things. First, no service is adopted that bills by usage without a hard free ceiling — a service that silently meters and invoices is worse than one that stops working, because a stopped feature is visible in an hour and a bill is visible in a month. Second, anything that costs money is not a background assumption but a named paid gap, carrying its manual workaround and its real price, listed in section 9.4. A new paid dependency requires a written waiver naming the amount, the trigger and the fallback.

RECURRING	TOTAL	LINES AT RISK
Rs 0 per month, infrastructure	₹67 domain, amortised monthly	8 gauges read every Friday

I-14 — ZERO RUPEES PER MONTH RECURRING

Three additional front ends, one more npm registry and three more hostnames add nothing to the monthly bill. Any new paid dependency requires a written waiver naming the amount, the trigger and the fallback. Enforcement: The AA-09 budget ledger is reviewed at the Friday gate; UptimeRobot and the free-tier threshold alerts are the tripwires.

The ledger below is the whole bill. Read the ALERT column as the operational contract: it is the number at which a human is told, not the number at which the service breaks. Every threshold sits at roughly seventy per cent of the free ceiling, which is deliberate — seventy per cent leaves a week of runway at pilot volumes, and a week is long enough to apply the degradation in section 9.3 without anybody improvising. AA-00 section 0.13 states the same cost line at programme level; this is the per-service breakdown behind it.

SERVICE	PLAN	WHAT IT COVERS NOW	FREE CEILING	COST	ALERT THRESHOLD
Cloudflare Pages	Free	Five sites: www, admin, seller, fleet and the existing console redirect	500 builds/month across all projects, unlimited bandwidth	₹0	Builds above 350/month. Four repos pushing means roughly 120 builds in a busy week.
Cloudflare DNS, SSL, WAF	Free	Three new subdomains, universal SSL, five WAF rules	Unlimited records; 5 custom WAF rules on free	₹0	A sixth WAF rule needed.
Google Cloud Run	Always-free	One backend service, four namespaces	180k vCPU-s, 360k GiB-s, 2 M requests/month	₹0	Alert at 140k vCPU-s or 1.6 M requests. Scale-to-zero stays on.
Neon Postgres	Free	71 tables, one primary branch plus one migration-rehearsal branch	0.5 GB storage, 190 compute-hours/month	₹0	Alert at 350 MB. audit_log archival cron is the release valve.
Cloudinary	Free	Product media, KYC documents, POD photos, advisory media	25 monthly credits ≈ 25 GB storage or bandwidth	₹0	Alert at 17 credits. KYC and POD are the new consumers; both are client-compressed.
GitHub Actions	Free (private)	CI for five repos, plus 11 scheduled cron workflows	2 000 minutes/month on private repos	₹0	Alert at 1 400 minutes. Five repos with cached installs run about 3 minutes each.
GitHub Packages	Free (private)	@atavishaala/kit and @atavishaala/contract	500 MB storage, 1 GB transfer/month on free	₹0	Alert at 350 MB. Prune pre-release versions monthly.
Cloudflare R2	Free	Cold archive of audit_log rows older than 180 days	10 GB storage, 1 M Class-A operations/month	₹0	Alert at 7 GB. Gzipped NDJSON is roughly 40:1 on audit rows.

The eight lines below are the ones consumed by usage rather than by builds, which is why seven of the eight Friday gauges are drawn from them.

SERVICE	PLAN	WHAT IT COVERS NOW	FREE CEILING	COST	ALERT THRESHOLD
WhatsApp Cloud API	Free tier	Staff invites, partner and fleet invites, rider OTP	1 000 service conversations/month	₹0	Alert at 700. Business-initiated template messages are billed; invites are the risk.
2Factor.in	Prepaid balance	SMS fallback for rider OTP where WhatsApp fails	Existing balance, roughly ₹0.20 per SMS	₹0 incremental	Alert when the balance drops below 500 credits.
Brevo	Free	Staff, partner and fleet invitation and password-reset email	300 emails/day	₹0	Alert at 200/day. Invites are bursty during onboarding weeks.
Sentry	Free (developer)	Four projects: api, admin, supplier, fleet	5 000 errors/month across the organisation	₹0	Alert at 3 500. Three new front ends will consume this faster than expected.
PostHog	Free cloud	Product analytics on all three portals	1 M events/month	₹0	Alert at 700k. Do not autocapture on the admin portal.
UptimeRobot	Free	Five monitors: api health, three portal roots, one deep API check	50 monitors at 5-minute intervals	₹0	No practical threshold.
Razorpay	Standard	Unchanged. No new integration for the portals.	Per-transaction, no monthly fee	₹0 fixed	Not applicable.
Domain atavishaala.in	Annual	Covers all subdomains including the three new ones	One registration	₹67 amortised	Renewal reminder at 60 days.

9.2 WHAT THREE MORE FRONT ENDS ACTUALLY ADD

The polling arithmetic, worked out loud, and the assumption that breaks it

Three portals add three things and only three. They add Cloudflare Pages projects, which cost nothing and are not metered on requests or bandwidth — the only meter there is builds, 500 a month across all projects, and four repositories pushing means roughly 120 builds in a busy week. They add Cloud Run requests, almost all of them from polling rather than from human clicks. And they add Neon compute hours, because a database that is being polled is a database that is awake.

The polling is the number that matters, so here is the arithmetic in full. Cut C-1 removed the fleet SSE offer stream and its stated fallback is 10 s polling everywhere. So the fleet offer inbox (FL-05) and the control room (FL-04) poll at 10 s, and the admin exception board (AD-11) and dashboard (AD-04) poll at 20 s. Assume six concurrent fleet dispatchers and four concurrent admin operators during a twelve-hour working day, 08:00 to 20:00.

```

fleet dispatch  6 seats x (3600 / 10) = 2 160 req/hour x 12 h = 25 920 req/day
admin ops       4 seats x (3600 / 20) = 720 req/hour  x 12 h = 8 640 req/day
supplier portal  2 orgs  x (3600 / 60) = 120 req/hour  x 10 h = 1 200 req/day
rider app       8 riders x (3600 / 30) = 960 req/hour  x 10 h = 9 600 req/day
polling subtotal = 45 360 req/day
human clicks, mobile app traffic, webhooks, cron calls (measured baseline) ~ 6 000 req/day
-----
total ~ 51 000 req/day x 30 = 1.53 M req/month
Cloud Run always-free: 2 000 000 requests/month. Alert threshold: 1 600 000. Headroom: 23 %.
```

That is uncomfortable, and it should be read as such. The estimate lands at roughly 76 per cent of the always-free request allowance and only 4 per cent below the alert threshold, on an assumption of ten concurrent desk seats. The assumption is the whole estimate. Twelve dispatchers instead of six puts the total past 2 M and the service starts billing per request. A dispatcher who leaves the control room open overnight adds 4 320 requests a day on their own, which is why the poll is suspended when the tab loses visibility — `document.visibilityState` is checked in the kit polling hook, and that single line is worth more headroom than any other optimisation in this document.

The second consumer is quieter and harder to control. Neon on the free tier autosuspends an idle compute after five minutes and bills compute-hours only while it is awake. A 10 s poll means it never idles for five minutes between 08:00 and 20:00, so the correct planning figure is not "requests times cost" but "twelve awake hours a day, every working day". Twelve hours times twenty-two working days is 264 compute-hours a month against a ceiling of 190. The free tier does not absorb that.

The mitigation is that only one of the four polling surfaces needs the database. The admin dashboard tile query (AA-05 section 5.12) is cached 30 s at the Cloudflare edge and never reaches Postgres on a cache hit; the fleet offer inbox is served from a 5 s in-memory cache in the Cloud Run instance keyed on `fleet_org_id`, so ten

dispatchers in one fleet produce one query rather than ten. With those two in place the measured awake window collapses to roughly seven hours a day, or 154 compute-hours a month, which fits with 19 per cent spare. Both caches are therefore not optimisations. They are the reason the free tier holds, and removing either one breaks I-14.

WHAT BREAKS THIS ESTIMATE

Concurrency, not volume. Order count barely moves the number; seats do. The tripwire is R-06 in AA-10: monthly vCPU-seconds above 140 000 or requests above 1.6 M. If it fires, the response is already chosen in section 9.3 and needs no design work.

WHAT WE HAVE NOT DONE

None of this is measured. There is no load test in this programme (section 9.12), so every figure above is arithmetic over an assumed seat count and an assumed 6 000 requests a day of baseline traffic. Treat it as a budget to check against reality in week one of the pilot, not as a result.

9.3 THE ALERT THRESHOLDS

The number, the channel, and the degradation that was agreed before it was needed

A threshold with no channel is a number in a document. A channel with no pre-agreed response is a message that starts a design meeting at the worst possible moment. So every gauge below carries three things: the number at which somebody is told, how they are told, and exactly what is done. The degradation is decided now, in writing, precisely so that nobody is designing under pressure with a meter running.

The reading mechanism is the 0 6 * * * free-tier-probe workflow. Where a provider exposes a usage API it is read and compared; where it does not — Cloudflare Pages builds, WhatsApp conversations, Brevo daily sends — the probe records what it can and the gauge is read by hand at the Friday 17:00 budget check, fifteen minutes, technical lead plus BE-1. Any gauge above 70 per cent gets a named action before Monday. That is the entire governance process and it is deliberately one meeting.

GAUGE	TOLD AT	CHANNEL	PRE-AGREED DEGRADATION — APPLIED WITHOUT FURTHER DISCUSSION
Cloud Run requests	1.6 M / mo	free-tier-probe -> WhatsApp to lead	Raise the fleet offer poll from 10 s to 30 s. This is cut C-1 fallback anyway, so it costs dispatch a few seconds of offer latency and nothing else. Then raise the admin board from 20 s to 60 s and extend the dashboard edge cache from 30 s to 120 s.
Cloud Run vCPU-seconds	140k / mo	free-tier-probe -> WhatsApp to lead	Same poll changes, plus turn on edge caching for the read-heavy list endpoints named in R-06: GET /admin/orders, GET /admin/jobs, GET /partner/products. All three are already ETag-tagged.
Neon storage	350 MB	free-tier-probe -> WhatsApp to lead	Run audit-archive manually rather than waiting for the first of the month, then VACUUM. If still above 350 MB, shorten the archive window from 180 days to 90. RB-09 is the procedure.
Neon compute-hours	150 h / mo	free-tier-probe -> WhatsApp to lead	Cut the working-day polling window from 08:00-20:00 to 09:00-19:00 by refusing poll requests outside it with 204, and drop the supplier portal poll entirely — two suppliers can press refresh.
Cloudinary credits	17 of 25	free-tier-probe -> WhatsApp to lead	Take cut C-6 for the rest of the month: POD is OTP plus geofence, no photo. Tighten client compression from 1600 px to 1200 px on KYC uploads. RB-10 is the procedure.
GitHub Actions minutes	1 400 of 2 000	Actions billing email + probe	Drop the scheduled workflows from eleven to six by merging offer-sweep, sla-watch and stale-shipment-alert into one 5-minute tick that calls three routes in sequence. Then disable CI on draft pull requests.
GitHub Packages storage	350 MB of 500	Manual, Friday check	Prune every pre-release version of @atavishaala/kit and @atavishaala/contract older than two weeks. Retain the last three release versions only.
Sentry errors	3 500 of 5 000	Sentry quota email	Sample the noisiest issue group to 10 per cent by fingerprint in beforeSend, and set the supplier-portal project to errors only with no breadcrumb capture. Never disable the api project.
PostHog events	700k of 1 M	PostHog usage email	Autocapture is already off on admin. Turn it off on supplier and fleet too, leaving only the nine named funnel events. Portal analytics is the first thing to lose, always.
WhatsApp conversations	700 of 1 000	Manual, Friday check	Invitations move to Brevo email only. Rider OTP stays on WhatsApp because SMS costs ₹0.20 a message and OTP delivery is on the critical path of a delivery.
Brevo email	200 of 300 / day	Brevo dashboard, Friday check	Batch onboarding invites across days rather than sending a cohort at once. Password resets are never batched.
2Factor.in balance	below 500 credits	free-tier-probe -> WhatsApp to lead	Top up. This is the one gauge whose response is to spend money — roughly ₹100 buys 500 SMS — and it is pre-approved because the alternative is a rider who cannot complete a delivery.

GAUGE	TOLD AT	CHANNEL	PRE-AGREED DEGRADATION — APPLIED WITHOUT FURTHER DISCUSSION
Cloudflare R2	7 GB of 10	Manual, monthly after audit-archive	Increase the gzip level on the NDJSON export and drop the diff payload from archived audit rows, keeping actor, action, target and timestamp. Archived diffs are the least-read bytes we hold.
Cloudflare Pages builds	350 of 500 / mo	Manual, Friday check	Stop building preview deployments for every branch; build main and open pull requests only.
Domain renewal	60 days out	Registrar email + calendar	Pay it. ₹800 a year is the entire infrastructure budget and losing the domain takes down four hostnames and every JWT issuer and audience claim in ENV_API.
UptimeRobot	no threshold	n/a	Five monitors against a ceiling of fifty. There is no plausible path to breaching this, so it carries no degradation.

The technical lead decides. Not because a single decision-maker is better governance, but because with four engineers an escalation tree is a delay dressed as a process — the same reasoning AA-08 section 8.14 uses for security incidents. What the lead is deciding is narrow: whether the trigger is real and whether to apply the written degradation or to accept the overage for a stated number of days. Choosing a different degradation is allowed, but it is a change to this table and it gets written down before it is applied, not after.

ORDERING, SO TWO PEOPLE DO NOT HAVE TO NEGOTIATE IT AT 22:00

When more than one gauge trips at once, degrade in this order: portal analytics first (PostHog), then poll intervals, then edge cache windows, then POD photo capture (C-6), then the supplier portal poll, and Sentry on the api project last. Nothing in that list touches money movement, audit logging, masking or the 240-minute manifest window. Those four are not degradable at any threshold and are not listed as options anywhere in this document.

9.4 THE PAID GAPS

Five things that cost money, their workarounds, and the signal that should make you buy

These five are not missing features. They are capabilities the system genuinely needs and does not have, each with a working substitute that is worse in a way we can describe. Naming them with real prices is what I-14 requires: the rule is not "never spend money", it is "never spend money by accident". Every row below is a purchase decision somebody can make in an afternoon with a number in front of them.

THE GAP	WHY IT IS NOT FREE	THE WORKAROUND WE SHIP INSTEAD	PAID OPTION AND PRICE
Masked click-to-call between rider and customer	No free provider offers a call bridge with number masking in India.	Time-boxed address reveal on the manifest, logged in pii_disclosures. Customer contact happens through a WhatsApp template send triggered by the rider, not a phone call.	Exotel or Kaleyra number masking, roughly ₹0.60/minute plus a ₹1 500/month platform fee.
Automated bank-account verification (penny drop)	Razorpay penny-drop and similar APIs are billed per verification.	A staff member transfers ₹1 by UPI and matches the holder name, then sets verified_at. Two suppliers and one fleet is roughly ten minutes of work per week.	Cashfree or Razorpay account validation, roughly ₹3 per verification.
Real-time rider GPS tracking on a map for the customer	Continuous location ingest is not free at any meaningful volume, and Maps tiles are billed.	Status milestones only: HANDED_OVER, IN_TRANSIT, OUT_FOR_DELIVERY. The POD captures a single geo point, which is all the geofence needs.	Google Maps Platform beyond the \$200 monthly credit, plus a location ingest path.
SMS delivery notifications at scale	Transactional SMS is billed per message and DLT registration is required.	WhatsApp template messages first, FCM push second, SMS only for OTP fallback.	2Factor.in or MSG91 at roughly ₹0.18 per transactional SMS plus DLT fees.
A hosted BI or reporting layer over the portal data	Metabase, Looker Studio against Postgres and similar all cost money or CPU we do not have.	Eleven fixed dashboard tiles computed by one SQL query, cached 30 s at the edge, plus masked CSV export for anything deeper.	Metabase Cloud from roughly \$85/month, or a small always-on VM.

9.4.1 The trigger for each — an operational signal, not a date

Masked click-to-call. Buy it when RB-03 fires more than four times in a week, or the first time a rider fails a delivery for "customer unreachable" on a shipment above ₹5 000. The workaround is a WhatsApp template send from the job screen, which depends on the customer reading WhatsApp within minutes; when that assumption breaks the failure mode is a returned order and a refund, and two refunds cost more than a month of the ₹1 500 platform fee. Note that buying it does not weaken the PII rule: a masked bridge means the rider still never sees the number, which is why this is the only paid gap that improves the privacy posture rather than trading against

it.

Penny-drop bank verification. Buy it at fifteen active payee organisations, not before. At two suppliers and one fleet the manual ₹1 UPI check plus holder-name match is ten minutes a week, and ₹3 a verification against a staff hour is not close. The real trigger is different and worth watching for: the first payout that fails at the bank because an account number was mistyped (RB-06). One of those costs a finance day of reconciliation and shakes a partner's confidence in the settlement ledger, and confidence in the ledger is the product.

Real-time rider GPS on a map. Do not buy this on customer request; buy it on dispute volume. The trigger is POD geofence disputes (RB-04) exceeding two per cent of deliveries, because at that point the single geo point the POD captures is no longer enough to settle an argument and a track is the only evidence. Customers asking to watch a dot move is a satisfaction issue that milestone notifications answer adequately at pilot scale.

SMS delivery notifications at scale. The trigger is a measured WhatsApp template delivery rate below 85 per cent for rider OTP over a full week. Below that, deliveries are failing for a reason that has nothing to do with the delivery, and ₹0.18 a message plus DLT registration is cheap. Above it, SMS is a second channel for the same job.

A hosted BI layer. The trigger is somebody asking for a number that the eleven fixed dashboard tiles and the masked CSV export cannot produce, twice in the same month, from two different people. One person with an unusual question writes SQL against a read replica branch in Neon. Two people with recurring questions is a reporting requirement, and \$85 a month is less than the standing cost of an engineer assembling numbers by hand. Be careful here: a BI tool with a live connection to this database sees unmasked columns, so buying it means an extra masked view layer and a decision about who holds those credentials. Budget the engineering, not just the subscription.

THE ONE THAT IS CLOSEST TO BEING BOUGHT

Masked click-to-call. R-04 in the risk register is "the rider needs the customer phone number" and the entire PII posture in AA-00 section 0.9 depends on never releasing it — GET /fleet/jobs/:id/manifest releases the delivery address only, job-scoped, inside a 240-minute window, writing a pii_dislosures row in the same transaction, and the phone is never in that payload. A call bridge is the only way to make the rider's job easier without breaking that. If any money is spent on this programme after go-live, it should be spent here first.

9.5 THE CRON INVENTORY

Eleven scheduled workflows, and why none of them is trusted to be on time

Cloud Run scales to zero, so there is no always-on process, so there is no background worker. Every scheduled thing in this system is a GitHub Actions workflow that makes one authenticated HTTP call to a route under /internal/cron/*, presenting CRON_SHARED_SECRET in a header. Actions is the clock; Cloud Run is the worker, because that is where the database credential lives. AA-02 section 2.7 has the design; this is the operational side of it.

SCHEDULE	WORKFLOW	WHAT IT DOES
*2 ****	offer-sweep	Expire lapsed job_offers, advance to the next wave, or mark UNASSIGNED.
*5 ****	job-queue-drain	Existing worker. Now also handles export jobs and webhook retries.
*10 ****	sla-watch	Advisory SLA breach alerts and shipment ready_by breach alerts.
0 ****	stale-shipment-alert	Shipments in AWAITING_ACCEPTANCE for more than 4 business hours.
30 2 ***	accrual	Accrue partner_payouts and fleet_payouts for the previous day's deliveries.
0 3 ***	doc-expiry-watch	Rider licence and vehicle insurance expiring within 30 days.
0 4 ** 1	payout-draft	Assemble the DRAFT partner and fleet runs for the previous week.
0 5 ** 1	cod-period-open	Open the new COD period and close declarations on the previous one.
0 6 ***	free-tier-probe	Read usage from each provider where an API exists; alert at threshold.
0 7 1 **	audit-archive	Export audit_log older than 180 days to R2 as gzipped NDJSON, then delete.
15 */6 ***	idempotency-gc	Delete idempotency_keys past expires_at.

9.5.1 Three consequences of using Actions as a scheduler

GitHub scheduled workflows have a documented five-minute minimum granularity, are explicitly best-effort on a shared runner pool, and carry no exactly-once guarantee. On a free account a two-minute cron delivers roughly every three to five minutes and occasionally skips a slot under Monday-morning load. Three consequences follow, and each has a mitigation that is already in the design rather than a plan to add later.

First, the $\ast/2$ offer-sweep line is aspirational. Treat effective granularity as five minutes and worst case as ten. Mitigation: no deadline is an event the cron produces. `OFFER_WAVE_TTL_SECONDS` is 600, 600, 900 for waves one to three, stored as `offer_expires_at` on `job_offers` at creation time, and every read path evaluates it lazily — an offer past its expiry is not returned as open and an accept on it is 409 `OFFER_EXPIRED`, decided by comparing `now()` to the stored column inside the accepting transaction. Same pattern for `sla_due_at` on `advisory_requests`. The cron only advances a wave so the next set of fleets is notified, and it is allowed to be late for that.

Second, a run can be skipped entirely. Mitigation: every job processes a backlog, not a tick. accrual accrues everything not yet accrued rather than "yesterday"; idempotency-gc deletes everything past `expires_at`; audit-archive exports everything older than 180 days. A missed run is therefore caught by the next one with no catch-up logic and no operator action, which is the only property that makes an unreliable scheduler acceptable.

Third, a run can fire twice or overlap with itself. Mitigation: every job is idempotent, and the two that mutate money are additionally guarded. `job-queue-drain` claims work with `SELECT ... FOR UPDATE SKIP LOCKED` inside a transaction, so two concurrent drains cannot claim the same row. accrual writes at most one `partner_payouts` and one `fleet_payouts` row per (delivery, period) and relies on a unique constraint to make a second attempt a no-op rather than a duplicate credit. Ledgers are append-only, so if a correction is ever needed it is a compensating row, never an `UPDATE`.

9.5.2 How you find out a cron stopped running at all

This is the real risk, and it is not the same as lateness. A workflow that fails loudly sends an Actions notification email to the repository watchers, which is BE-1 and the technical lead. A workflow that stops being scheduled at all sends nothing — and GitHub disables scheduled workflows on repositories with no activity for sixty days, which is a realistic state for `atavishaala-api` in a quiet month after go-live.

So the detection is a heartbeat, not a subscription. Every cron route writes a row to `cron_runs` (workflow, started_at, finished_at, rows_touched, outcome) as its last statement. The `0 6 * * * free-tier-probe` additionally checks that every one of the eleven workflows has a `cron_runs` row inside twice its nominal interval, and raises if any does not — so a dead $\ast/2$ sweep is reported within a day by a different workflow. The probe watching itself is the residual hole: if `free-tier-probe` is the workflow that dies, nothing reports it, and the only backstop is the Friday budget check noticing that no gauge reading arrived. That is a human control and it is admitted as one. The paid fix is Cloud Scheduler with a dead-man alert at roughly ₹30 a month for three jobs beyond the free three, which breaks I-14 and is not bought.

9.6 DEPLOYS

Five repositories, one ordering constraint, and an asymmetric rollback

The ordering constraint is the only thing in this section worth memorising: the API deploys first, always. It is safe to do so because the contract is additive — new fields, new endpoints, no removed or renamed response keys — which is the rule AA-01 section 1.9 enforces on `@atavishaala/contract`. An API ahead of a portal serves fields the portal ignores. A portal ahead of the API calls endpoints that return 404 and renders a broken screen to a supplier. The two states are not equally bad, so the order is not negotiable.

1. `atavishaala-api`. Merge to main. CI runs the twelve gates. On green, tag `vX.Y.Z`; the tag push builds the container, pushes it to Artifact Registry and deploys a new Cloud Run revision in `asia-south1` with traffic at 100 per cent. Migrations run as a pre-deploy step against `DIRECT_URL`, not the pooled `DATABASE_URL`. Wait for `GET /healthz` to report the new build sha before continuing.
2. `atavishaala-portal-kit`, only if the release touches it. Bump the version in `package.json`, add a dated `CHANGELOG.md` entry naming the task id, and tag from main. CI fails the publish workflow if the top `CHANGELOG` heading does not match the package version, so a forgotten entry is a red build and not a silent release. The package publishes privately to GitHub Packages under the `@atavishaala` scope.
3. `atavishaala-admin-portal`. Bump the `@atavishaala/kit` dependency to the version just published, merge to main, and Cloudflare Pages builds and promotes to `admin.atavishaala.in`. Deploy admin before the other two portals: it is the only surface where an operator can undo a mistake made by the other two.
4. `atavishaala-supplier-portal`, then `atavishaala-fleet-portal`. Same procedure, `seller.atavishaala.in` and `fleet.atavishaala.in`. Fleet last, because a rider mid-delivery is the least interruptible user in the system and a Pages promotion invalidates their loaded bundle.

5. Smoke: sign in to each portal, load its dashboard, and confirm the build sha in the footer matches what was just deployed. VITE_BUILD_SHA is injected by CI for exactly this purpose. Then check Sentry for a new issue group in the ten minutes after promotion.

9.6.1 Rollback is not symmetric, and pretending otherwise is the failure mode

A portal rollback is trivial. Cloudflare Pages retains previous deployments, and promoting one to production from the dashboard takes about ten seconds with no rebuild — no dependency on Actions minutes, no dependency on the package registry being reachable. Any of the four engineers can do it and it needs no approval. That is the fastest safety valve in the entire system and it is the reason front-end risk is tolerable.

An API rollback is not available in practice. Migrations are forward-only by policy: there are no down migrations, so revision N-1 may be asked to run against a schema it has never seen. Deploying the previous container is therefore a schema decision, not a deploy button, and treating it as a button is how a bad afternoon becomes a bad week.

So the API rollback procedure is forward-fix behind a flag. Every feature that spans a week boundary ships behind a flag defaulting to off — the mechanism is a `feature_flags` table read through a 60 s in-process cache, keyed by flag name with an optional `partner_org_id` or `fleet_org_id` scope, toggled from the admin portal under `staff.manage` with the change written to `audit_log` like any other mutation. It is a database row and not an environment variable on purpose: an env change on Cloud Run is a new revision, which is a deploy, which is the thing we are trying to avoid doing under pressure. Turning a flag off takes effect within 60 seconds with no deploy and no restart.

1. Confirm the fault is in the new revision, not in data: check Sentry for the issue group first seen at the deploy timestamp, and check whether the same request id appears in Cloud Logging before the deploy.
2. Turn off the flag that gates the faulty feature, from AD-40 or by a single UPDATE on `feature_flags` with a second person watching. Wait 60 seconds. Re-test.
3. If no flag gates it, deploy a forward fix. Cherry-pick the revert commit onto main, tag, and let CI deploy. Roughly eight minutes end to end with a warm cache.
4. Only if the fault is a migration itself: stop, page the technical lead, and treat it as RB-11. Restoring a Neon branch is a data decision with an announced outage, never a quiet rollback.

THE GAP THIS LEAVES

There is no canary and no traffic splitting. Cloud Run supports revision traffic splitting and it is free, but using it means two revisions running against one schema at once, which is only safe if every migration is backward-compatible with the previous code — a property this programme does not guarantee because M4 explicitly is not. So deploys are all-at-once, and the mitigation is that they happen at 11:00 on a weekday with someone watching Sentry, never on a Friday afternoon and never after 16:00.

9.7 THE ELEVEN RUNBOOKS

Written as steps because reasoning belongs in the other eleven sections, not here

Each runbook below is written for somebody who is tired and possibly not the person who built the thing that broke. There is no reasoning inside a procedure step: if you want to know why the offer sweep cannot be trusted, that is section 9.5, and reading it at 02:00 is not the plan. Every runbook starts with a verification step, because the most common incident is a symptom that is not real, and ends with a record-keeping step, because an incident that leaves no row behind cannot be counted against the triggers in section 9.4. D-20 requires each of these to have been rehearsed once by somebody other than its author before go-live.

ID	SYMPTOM	OWNER	FIRST MOVE
RB-01	A job is stuck UNASSIGNED	Ops	Rebroadcast at wave ALL
RB-02	A supplier misses ready_by	Ops	Call the supplier
RB-03	A rider cannot reach the customer	Ops	Trigger the WhatsApp template from the job screen
RB-04	POD geo is outside the fence	Ops	Flag the POD, hold the fleet payout line, ask the fleet for an explanation within 24 h
RB-05	COD variance above 2 %	Finance	Block the fleet from offers, open a dispute, move the fleet to prepaid-only until reconciled
RB-06	A payout run failed at the bank	Finance	Mark the run FAILED, do not re-release

ID	SYMPTOM	OWNER	FIRST MOVE
RB-07	A partner disputes a settlement figure	Finance	Answer from partner_ledger and the pricing_snapshot
RB-08	A staff account is compromised	You	Suspend the staff user, revoke every refresh family, read the audit log and disclosure register for the session window
RB-09	Neon storage crosses 350 MB	BE-1	Run audit-archive manually, then VACUUM
RB-10	Cloudinary credits cross 17	BE-1	Drop POD photo capture (C-6) for the rest of the month
RB-11	M4 rollback	BE-2	Restore the Neon branch taken before the read-only window

RB-01 A job is stuck UNASSIGNED · owner Ops

1. Open AD-11 /jobs/unassigned and find the job. Read the wave history: confirm all three waves actually broadcast and lapsed rather than the job never having been offered at all.
2. If no wave ever broadcast, the shipment is not READY_FOR_PICKUP. Open AD-08 and check the shipment state; a supplier who has not marked ready is RB-02, not RB-01.
3. Check serviceability on AD-14 for the drop pincode. If no fleet covers it, skip to step 6 — rebroadcast will not help.
4. Rebroadcast at wave ALL from the inline action on AD-11. This offers to every eligible fleet at once rather than by tier.
5. Wait 20 minutes. Do not rebroadcast again in that window; a second broadcast resets countdowns and makes the wave history unreadable.
6. If still unassigned, assign ATV-SELF from the same screen. ATV-SELF is the internal fleet seeded by M5 and exists for exactly this.
7. Add an internal note on AD-06 naming the pincode and the reason. Count it: three of these in a week for the same pincode is a service-area gap, not an incident.

RB-02 A supplier misses ready_by · owner Ops

1. Open AD-07 and read the ready_by breach lane. Confirm the breach is real: the shipment is still in ACCEPTED or PACKING and ready_by is in the past.
2. Call the supplier on the support contact from AD-22. Ask for one thing only: a new ready_by timestamp they will commit to.
3. If they commit, record the new ready_by from AD-08 and leave the shipment alone. The on-time metric still counts the original breach; that is intended.
4. If they do not commit, or do not answer within 30 minutes, cancel the shipment from AD-08. Cancelling one shipment does not cancel the order — orders.status is derived, and a two-supplier order becomes PARTIALLY_DELIVERED or CANCELLED by reduceOrderStatus() on its own.
5. Confirm the refund initiated for the cancelled lines. If it needs approval it appears on AD-35, where initiator and approver are never the same staff user.
6. Flag the breach against the supplier on AD-21 so it lands in the on-time rate. Three breaches in a week is a partner conversation, not an ops action.

RB-03 A rider cannot reach the customer · owner Ops

1. Confirm the rider has actually opened the manifest. Open AD-12 and check the disclosure entry; if there is none, the rider is looking at a masked address and the problem is that they never pressed Reveal on FL-07.
2. Check the manifest window. It is 240 minutes from OUT_FOR_DELIVERY. An expired window means the address is masked again and the rider needs the job re-opened, not the customer called.
3. Trigger the WhatsApp template to the customer from the job screen. Do not read the number to anybody. No screen, no role and no permission in this system renders more than the last four characters, and there is no reveal affordance to ask for.
4. Record a failed attempt on FL-10 with the outcome code, or ask the fleet dispatcher to. Three recorded attempts force RTO automatically and refund the customer on RTO_DELIVERED.
5. If the customer replies with a correction, update the drop details from AD-06 and ask the fleet to re-open the job. Do not edit the address to something the customer did not say.
6. Note the attempt on the order. If this runbook fires more than four times in a week, that is the trigger in section 9.4 for buying masked click-to-call.

RB-04 POD geo is outside the fence · owner Ops

1. Open AD-13 /pod-review. Read the geo distance on the POD. POD_GEOFENCE_METRES is 200, so anything the screen surfaces is already outside it.
2. Check for the innocent explanation first: a flat block with a poor GPS fix, or a pincode centroid used because the device denied location. A distance under about 500 m with an OTP that verified is almost always this.
3. Flag the POD from AD-13. Flagging holds the fleet payout line for that job only; it does not block the fleet from offers and it does not touch the customer refund path.
4. Ask the fleet for a written explanation within 24 hours, from the fleet detail screen AD-26. Name the job id and the distance in the request.
5. On a satisfactory answer, clear the flag and release the payout line. On no answer within 24 hours, leave it withheld — the supplier and fleet payout screens show withheld lines with their reason, so the fleet can see why.
6. If the same rider produces three flagged PODs, suspend the rider from AD-30 and tell the fleet. A pattern is a rider problem; one point is a phone problem.

RB-05 COD variance above 2 % · owner Finance

1. Open AD-38 /cod and read expected against declared against received for the open period. COD_VARIANCE_BLOCK_BPS is 200 basis points, so the block threshold is 2 per cent of expected cash.
2. Confirm the variance is cash and not timing: an undeclared remittance in transit looks identical to missing money on the morning it is banked. Check the age buckets before doing anything else.
3. Block the fleet from receiving offers, from AD-25. This is the containment step and it is reversible; do it before the conversation, not after.
4. Open a dispute against the period from AD-38, with the variance figure in paise. All money in this system is integer paise in *_paise columns; do not restate it in rupees in the dispute note.
5. Move the fleet to prepaid-only for new jobs until the period reconciles. That is a tier change on AD-26, not a ledger operation.
6. When cash arrives, post the receipt. The fleet_ledger is append-only: a correction is a compensating entry, never an edit, and there is no screen that offers an edit.
7. Record the outcome against the period. R-09 in AA-10 tracks this; two blocked periods for one fleet is a commercial decision, not an operations one.

RB-06 A payout run failed at the bank · owner Finance

1. Open AD-37 /payouts and the run. Establish which lines the bank rejected and which settled. A partial failure is the normal case and the whole run is not wrong.
2. Mark the run FAILED. Do not re-release it and do not retry it. A released run has a UTR captured against it, and re-releasing is how one payee gets paid twice.
3. Create a new run for the same period from AD-37. The accrual rows are still there; the new run assembles from the same accruals and excludes the lines that settled.
4. Note the bank reason verbatim on the failed run. If the cause is a wrong account number, the payee organisation fixes it themselves on their own bank screen, which pauses their payouts until it is re-verified.
5. Reconcile to the paisa before releasing the new run. Gross minus commission minus TDS equals net on every line, and the run total equals the sum of the lines. If it does not, stop and call BE-1.
6. Log the incident. A payout failure caused by a mistyped account number is the trigger in section 9.4 for buying penny-drop verification.

RB-07 A partner disputes a settlement figure · owner Finance

1. Get the run id and the line the partner disputes. Open AD-37, then the run detail, then the line.
2. Answer from two sources and only two: partner_ledger for what was credited and debited, and the pricing_snapshot attached to the order line for what the commission percentage was at the time of sale. The current rate card is not evidence about a past order.
3. If the ledger and the snapshot agree with each other, the figure is right. Send the partner the line-by-line breakdown from their own AD-22 activity view; they can see the same numbers.
4. If they disagree, the snapshot wins and the ledger is corrected. Post a compensating entry — never edit a ledger row. The append-only Postgres rules installed by M7 will reject an UPDATE anyway.
5. Include the compensating entry in the next run rather than releasing an off-cycle payment.
6. Write the reason on the entry. A dispute that produces a compensating entry with no readable reason is the same dispute again next month.

RB-08 A staff account is compromised · owner technical lead

1. Confirm compromise before containing: open AD-41 and read the audit tab for that staff user. Look for a sign-in from an unexpected hour, then for mutations they would not normally make.
2. Suspend the staff user from AD-40. Suspension is immediate and blocks the next token refresh.
3. Revoke every refresh family for that user with POST /admin/auth/logout-all. Suspension alone leaves a live access token valid for up to 900 seconds on a portal.
4. Call the technical lead. One name, no rota, no escalation tree — with four engineers a tree is a delay dressed as a process. AA-08 section 8.14 is the full incident procedure and this runbook is its checklist.
5. Read both registers for the session window: audit_log for what was changed, and the disclosure tab of AD-41 for every pii_disclosures row the session produced. Each disclosure is one delivery address released for one job. Count them and list the job ids.
6. Decide on rotation with the lead. Rotating PII_ENC_KEY requires a re-encrypt job and is not a five-minute action (AA-08 gap G-01); rotating the JWT signing key logs every user out of every surface. Neither is automatic.
7. Re-enrol TOTP for the user before restoring access, and write the incident up the same day. Notify affected customers if any address was disclosed outside a legitimate delivery.

RB-09 Neon storage crosses 350 MB · owner BE-1

1. Confirm the figure in the Neon console rather than trusting the probe. Then find where it went: audit_log and pii_disclosures are the only tables that grow without bound, and audit_log is almost always the answer because it carries a before/after diff up to AUDIT_DIFF_MAX_BYTES = 8192 per row.
2. Run the audit-archive job manually rather than waiting for 0 7 on the first. It exports rows older than 180 days to R2 as gzipped NDJSON and then deletes them.
3. VACUUM the table. A delete without a vacuum does not return space to the free-tier storage figure, which is the single most common way this runbook appears not to have worked.
4. Re-read the figure. If still above 350 MB, shorten the archive window from 180 days to 90 and run again. Ninety days still covers the DPDP-relevant retention question and it is a config change, not a migration.

1. If still high after 90 days, look at webhook_deliveries and idempotency_keys. Both have a GC path (15 */6 for idempotency-gc) and both can be trimmed harder without losing anything auditable.
2. Record the pre and post figures against the date. Two of these in one month means the 0.5 GB tier is genuinely too small and the answer is an archival schedule change, not another cleanup.

RB-10 Cloudinary credits cross 17 · owner BE-1

1. Read the credit figure in the Cloudinary console and split it: storage credits and bandwidth credits are consumed differently and the fix is different.
2. If bandwidth, check that portals are requesting transformed derivatives and not originals. A KYC document rendered at full resolution in AD-24 is the usual cause and it is a one-line URL fix.
3. If storage, take cut C-6 for the remainder of the month: POD becomes OTP plus geofence with no photo. This is a flag, so it takes effect within 60 seconds and does not need a deploy.
4. Tighten client-side compression on the remaining upload paths from 1600 px to 1200 px longest edge. KYC documents stay legible; POD photos were the largest consumer and are now off.
5. Tell ops that dispute evidence is weaker until the month rolls over, so RB-04 will be harder to settle. That consequence is stated in C-6 and it is real.
6. Turn the photo flag back on the day the credits reset, and write down the date. A cut that silently becomes permanent is how the geofence ends up being the only evidence forever.

RB-11 M4 rollback · owner BE-2 · announced outage

This is the only runbook that is a data operation, and it is the only one that must not be run alone. M4 is the shipment split: it backfills one shipment per existing order inside a single transaction, copies status through a mapping table and writes synthetic order_events from existing timestamps. If it is wrong, the marketplace has the wrong order states and no down migration exists.

1. Stop writes. Put the API into the read-only window flag; the app and all three portals render a maintenance state and mutating routes return 503.
2. Confirm the failure is structural and not a handful of rows. A derived-status diff of a few orders is fixed by calling recomputeOrder(orderId), not by restoring a database.
3. Announce a 20-minute outage to the pilot suppliers and the fleet before touching anything. This is step three and not step eight on purpose.
4. Restore the Neon branch taken immediately before the read-only window opened. The branch is created as part of the M4 procedure and its name carries the timestamp.
5. Point the app at orders.legacy_status. The column is retained by M4 for exactly this and the mobile build already reads it as a fallback.
6. Verify: derived-status diff against the restored snapshot must be zero rows (D-18). Run it before lifting the read-only flag, not after.
7. Lift the flag, confirm one live order transitions end to end, and write up what the backfill got wrong before attempting M4 again. A second attempt without a written cause is the same outage.

WHAT IS DELIBERATELY NOT A RUNBOOK

A direct UPDATE in the Neon console. It is possible, nothing in the design prevents it, the audit trail will not carry it, and recomputeOrder() will overwrite a hand-edited orders.status on the next shipment transition anyway. The control is procedural: a console write requires a second person present and a note in the ops log naming the statement. That is a process control and it is admitted as one rather than described as a safeguard.

9.8 OBSERVABILITY

Diagnosable, not observable, and where you look first

The honest framing from AA-02 section 2.10 holds: this system answers "what happened to this request" well and "is the system healthy right now" badly. There is no APM, no tracing, no metrics backend and no error-rate alert. What exists is four tools, each on a free tier, joined by one key — X-Request-Id, echoed on every response, rendered in the portal error toast, and set as a Sentry tag. A support conversation that produces a request id finds both the exception and the log lines around it, and that single join is most of the operational value here.

TOOL	WHAT IT CARRIES	FREE-TIER LIMIT	WHAT HAPPENS WHEN IT IS HIT
Sentry	Exceptions from four projects: api, admin, supplier, fleet. beforeSend runs the same masking helper the serialiser uses over body, headers, breadcrumbs and extra context, drops the event if it cannot parse what it is scrubbing, and strips local variable capture.	5 000 errors/month, alert at 3 500	Sentry silently discards over quota for the rest of the month. You lose visibility without being told, which is why the threshold is at 70 per cent and the degradation samples the noisiest fingerprint rather than waiting.

TOOL	WHAT IT CARRIES	FREE-TIER LIMIT	WHAT HAPPENS WHEN IT IS HIT
Pino to Cloud Logging	Structured JSON, one line per request, with request id, actor kind and tenant id bound at the chain. Redaction paths are set once at logger construction: authorization, cookie, x-idempotency-key, otp, otp_hash, totp, phone, email, address, account_number, ifsc, upi_handle, gstin, pan, aadhaar, and res.body.manifest.address.	Cloud Logging free tier, 30-day retention	Nothing breaks; logs age out at 30 days. Anything needed beyond that must have been exported. Cloud Run stdout is the entire logging stack — no agent, no shipper.
PostHog	Nine named funnel events on the supplier and fleet portals. No autocapture on admin, because admin is a PII-adjacent surface and autocapture records DOM text.	1 M events/month, alert at 700k	Events are dropped. Portal analytics is the first thing degraded under any threshold, in every ordering in section 9.3.
UptimeRobot	Five monitors at 5-minute intervals: GET /healthz on the api, the three portal roots, and one deep API check that exercises a database read.	50 monitors, 5-minute interval	No plausible breach. The real limitation is the interval: a two-minute outage is invisible, and a 30 per cent failure rate that does not take the process down is invisible entirely.

9.8.1 Four numbers every morning, two dashboards that answer the real question

The morning check is four numbers and it takes under five minutes. One: unassigned jobs on AD-11 — anything older than an hour is RB-01. Two: ready_by breaches in the AD-07 breach lane — that is RB-02 and it is the number a supplier will be asked about. Three: new Sentry issue groups in the last 24 hours on the api project, ignoring the portals for this purpose. Four: COD exposure on AD-38, read as an amount and an age, because cash that is old is cash that is gone.

Two dashboards answer "is the marketplace working". AD-04, the operations dashboard, has eleven tiles computed by one SQL query cached 30 s at the edge, and it answers the demand side: orders flowing, shipments accepted, moderation backlog, advisory SLA. AD-07, the shipment board, answers the fulfilment side, and it is the ops home screen for a reason — a Kanban with a breach lane makes "nothing is moving" visible in a way a number cannot. If both are green and the four numbers are flat, the marketplace is working, and no other screen will tell you differently.

9.9 ON-CALL, SUCH AS IT IS

Four engineers, no rota, and the specific risk that creates

There is no on-call rota. Four engineers cannot staff one, and pretending otherwise produces a schedule nobody honours, which is worse than an honest gap because it removes the pressure to design around the gap. So the posture is stated plainly: business-hours response, 09:00 to 20:00, for anything that is not money movement or data loss. Outside those hours the system is unattended.

SEVERITY	WHAT QUALIFIES	NAMED FIRST RESPONDER	RESPONSE
S1	Money moved wrongly, cash unaccounted for, data loss, a suspected staff compromise, or the api returning errors on every request.	Technical lead	Immediate, any hour, phone. This is the only severity that reaches anybody overnight, and it reaches one person. AA-08 section 8.14 governs if it is a security incident.
S2	A whole flow is broken but nothing is lost: offers not broadcasting, POD capture failing, a portal down, payouts blocked.	BE-2 for fulfilment, BE-1 for platform	Same business day. Containment first — a flag off is better than a fix at 19:00 — then a forward fix the next morning.
S3	Anything one operator can work around: a stuck job, a wrong figure on a screen, a slow list.	Operations associate, then the module owner	Next business day, through the ordinary backlog.

Escalation is one hop. An operations associate who cannot resolve something with the runbooks calls BE-1 or BE-2 depending on whether it is platform or fulfilment; either of them escalates to the technical lead. Nobody escalates twice, and there is no queue between the person with the problem and the person who can fix it.

What an operations associate is authorised to do without an engineer matters more than the tree, because it is what makes business-hours cover workable. Three actions are pre-authorised and none of them needs approval: pause offer broadcast for a fleet or a pincode from AD-25 and AD-14; block a fleet from receiving offers (the

containment step in RB-05); and close a disclosure window early on a job, which re-masks the address before the 240 minutes elapse. Each is reversible, each is audited, and each reduces exposure rather than increasing it. Every action that increases exposure or moves money needs a second person: refunds are already two-person on AD-35, and a payout release requires a UTR captured by finance.

THE RISK THIS CREATES, NAMED

A COD-heavy failure that starts at 21:00 is not seen until 09:00. Concretely: a fleet whose riders collect cash all evening against a variance that has already crossed 2 per cent keeps receiving offers for twelve hours, because the block in RB-05 is a human action and nothing automated applies it. Worst case is roughly one evening of one fleet's COD exposure, which at pilot volumes is small in rupees and large in trust. The cheap partial mitigation is deliberately not automated blocking — an automated block on a bad reading strands real deliveries — but a nightly cap: no fleet may hold more than a configured COD exposure at once, enforced at offer eligibility. That cap is not in this programme and is the first item in the operations backlog.

9.10 BACKUPS AND DATA RECOVERY

A 24-hour restore window, a weekly dump, and one gap with no second copy

Neon on the free tier gives point-in-time restore over a 24-hour history window. Within that window the recovery is genuinely good: you create a branch at any second in the last 24 hours and the branch is a full, writable copy at that instant, available in well under a minute because it is a copy-on-write operation rather than a data transfer. That is what makes RB-11 viable at all, and it is why the M4 procedure creates a named branch before opening the read-only window.

What it does not cover is everything older than 24 hours. A logical corruption discovered on Monday that began on Friday is outside the window, and the free tier has no answer for it. So the second line of defence is a weekly logical dump: the `pg_dump` runs from a GitHub Actions workflow on Sunday at 01:00 against `DIRECT_URL`, gzips the output, and writes it to Cloudflare R2 under `backups/pg/YYYY-WW.sql.gz` with a four-week retention. It is a custom-format dump so a single table can be restored without the rest.

SCENARIO	RPO / RTO	WHAT ACTUALLY HAPPENS
Bad migration or bad backfill, caught within 24 h	RPO seconds, RTO 20 min	Neon branch at the pre-migration timestamp. RB-11. The 20 minutes is almost entirely the announced outage and the zero-row derived-status diff, not the restore itself.
Accidental bulk delete, caught within 24 h	RPO seconds, RTO 30 min	Branch at a point before the statement, export the affected tables, reload into the live branch. Slower than a full restore because the rest of the database has moved on.
Corruption discovered after 24 h	RPO up to 7 days, RTO 2-4 h	The weekly dump. You lose up to a week of writes, which for money tables is unacceptable and for catalog tables is survivable. There is no better answer on the free tier.
Neon account or region loss	RPO up to 7 days, RTO 1 day	Restore the weekly dump into a fresh Neon project, repoint <code>DATABASE_URL</code> , redeploy. Never rehearsed. This is the least tested path in the entire programme.
Cloudinary asset loss	no recovery	There is no second copy. KYC documents, POD photos and product media exist once, in Cloudinary, on a free tier with no backup product.

One restore rehearsal is mandatory before go-live and it is not the branch operation — branching is exercised by the M4 rehearsal already. The rehearsal that matters is the cold one: take the most recent weekly dump from R2, load it into a new Neon project, point a local API at it, sign in to the admin portal against it, and confirm that a masked order list renders and a payout run reconciles. Time it and write the number down. Until that has been done once, the RTO figures above are estimates written by the person who also wrote the backup script, which is the weakest kind of estimate there is.

THE GAP WITH NO SECOND COPY, AND ITS WORKAROUND

Cloudinary free has no backup product and no export API worth relying on, so every KYC document, every POD photo and every product image exists exactly once. Losing the account loses the evidence base for every open dispute. The workaround is partial and it is chosen because it is free: the monthly audit-archive workflow also writes a manifest of every Cloudinary `public_id` referenced by `partner_kyc`, `fleet_kyc` and `pod_records` to R2, so after a loss we know precisely what is gone and can ask the two suppliers and the fleet to resubmit. It does not preserve the bytes and it is not a backup. Copying the bytes into R2 is roughly 8 GB of Class-A operations against a 10 GB ceiling that already holds the audit archive, so it does not fit today; it is the first thing to buy R2 capacity for.

9.11 GO-LIVE AND THE FIRST WEEK

Two suppliers, one third-party fleet plus ATV-SELF, and the order it happens in

The pilot is deliberately small enough that every failure is a conversation rather than a statistic: two product companies, one third-party delivery company, and ATV-SELF as the fallback fleet seeded by M5. Nothing about the sequence below is reversible in the sense of undoing history, so each step has a smoke check and the sequence stops at the first failed check rather than continuing to see whether it matters.

1. Apply M1 to M8 to production in order. Smoke: the derived-status diff is zero rows (D-18) and the first SUPER_ADMIN created from BOOTSTRAP_ADMIN_EMAIL can sign in and is forced to enrol TOTP. Then remove BOOTSTRAP_ADMIN_EMAIL from the environment.
2. Create the real staff seats on AD-40 and confirm the effective-permission drawer shows what each person should have, with DENY winning. Abort if any seat resolves to more than intended.
3. Invite the two product companies (AD-23), review their KYC (AD-24), verify bank accounts by the manual ₹1 UPI check, and take each to ACTIVE. Smoke: one partner-drafted product passes moderation and appears in the mobile app catalog (D-06).
4. Invite the fleet (AD-27), review KYC and insurance expiry (AD-28), define one service area with a rate card, add two riders. Smoke: an offer broadcasts to that fleet and appears on FL-05 with a countdown.
5. Run one end-to-end internal order with staff as the customer, paid online, single supplier. Smoke: shipment accepted, packed, ready, handed over, manifest opened (one pii_disclosures row written), OTP delivery completed inside 200 m (D-11).
6. Run one internal COD order to ATV-SELF. Smoke: the COD period opens, the declaration flows, and variance computes to zero (D-14).
7. Run one weekly payout cycle on internal orders only: accrue, draft, lock, release with a UTR, reconcile to the paisa (D-13). Abort go-live if this does not reconcile exactly.
8. Open to real customers on a single pincode covered by the fleet, then widen one pincode at a time.

TWO THINGS THAT MUST BE TRUE BEFORE ONE REAL CUSTOMER ORDER FLOWS THROUGH A SHIPMENT

First, the masking check: no screen in any of the three portals renders more than the last four characters of any masked field (D-04), verified by walking the supplier and fleet portals as a real user, not by reading code. Second, a released payout run that reconciles to the paisa on internal orders. Money correctness before volume, in that order, because a wrong ledger is not fixable by apologising.

ABORT CRITERIA — STOP AND DO NOT WIDEN

Any of: the derived-status diff is non-zero; a masked field renders in full anywhere; a payout run does not reconcile; the manifest disclosure does not write a pii_disclosures row in the same transaction; or Cloud Run requests in the first week extrapolate above 1.6 M a month before real customers are added.

9.11.1 The first-week watch list

NUMBER, CHECKED DAILY	WHERE	VALUE AT WHICH THE PILOT STOPS
Unassigned jobs older than one hour	AD-11 /jobs/unassigned	More than three in a day, or any one requiring ATV-SELF twice for the same pincode.
ready_by breaches	AD-07 breach lane	More than 15 per cent of shipments in a day. Below that it is a supplier conversation.
POD geofence flags	AD-13 /pod-review	More than 5 per cent of deliveries. Above that the geofence or the fleet is wrong, and both stop the pilot until it is known which.
COD variance on the open period	AD-38 /cod	Any variance above the 200 bps block threshold, or any period ageing past seven days undeclared.
Cloud Run requests, extrapolated monthly	Cloud Run metrics + free-tier-probe	Above 1.6 M a month. This stops widening, not the pilot: apply section 9.3 first and re-read the next day.

9.12 WHAT OPERATIONS CANNOT DO YET

Seven gaps, stated as gaps rather than as things that are almost done

Every item below is a capability this operation does not have. None of them is worked around in a way that makes the gap disappear; each is worked around in a way that makes the gap survivable at pilot scale, which is a different claim. They are numbered so AA-10 can carry them into the risk register and so a Friday gate can ask about one of them by name.

ID	THE GAP	WHAT IT MEANS IN PRACTICE	WHAT WE DO INSTEAD, AND WHO OWNS IT
O-01	No SIEM, no anomaly detection	The PII disclosure register is not watched by anything. An operator who opens forty manifests in an hour produces forty legitimate-looking rows and no alert. This is AA-08 gap G-05 seen from the operations side.	A cron produces a report and a human reads it: disclosure counts per staff and per fleet user, weekly, at the Friday gate. Detection latency is up to seven days. Technical lead.

ID	THE GAP	WHAT IT MEANS IN PRACTICE	WHAT WE DO INSTEAD, AND WHO OWNS IT
O-02	No synthetic transaction monitoring	UptimeRobot proves the api answers /healthz and the portals serve HTML. It proves nothing about whether a customer can complete a checkout. A broken checkout is discovered by a customer.	One of the five monitors is a deep check that exercises a database read, which catches a dead connection pool. Everything past that is unmonitored. BE-1.
O-03	No load test	The polling arithmetic in section 9.2 is arithmetic. Nobody has driven ten concurrent dispatch seats against this api and measured vCPU-seconds, so the Cloud Run headroom figure of 23 per cent is a calculation, not a result.	First-week measurement replaces it: read actual requests and vCPU-seconds daily and compare with the model. If the model is wrong by more than 30 per cent, section 9.2 is rewritten. Technical lead.
O-04	No staging with production-like data volumes	The migration rehearsal branch holds a restored snapshot, which is correct data at pilot size. No environment anywhere holds a year of orders, so an index that is adequate at 5 000 rows and pathological at 5 M is not discoverable before production.	The index set is justified query by query in AA-03 section 3.10 and EXPLAIN output is reviewed at merge for every new list endpoint. That catches missing indexes, not bad plans at scale. BE-1.
O-05	No automated restore test	The weekly dump is written and never read. A dump that cannot be loaded is not a backup, and we would find out during the incident.	One manual cold-restore rehearsal before go-live (section 9.10) and a quarterly repeat in the calendar. Automating it needs somewhere to restore into. BE-1.
O-06	No automated COD exposure cap	A fleet can keep accepting COD jobs overnight while its variance is already past the block threshold, because the block in RB-05 is a human action and nobody is awake. Section 9.9 names this as the specific overnight risk.	Morning check on AD-38 as the first of the four daily numbers. The cap at offer eligibility is the first item in the operations backlog. BE-2.
O-07	No dead-man switch on the watcher	free-tier-probe verifies that the other ten cron workflows ran. Nothing verifies that free-tier-probe ran. GitHub also disables scheduled workflows after sixty days of repository inactivity.	The Friday budget check notices a missing gauge reading. That is a human control with a one-week detection latency. Cloud Scheduler with a dead-man alert is roughly ₹30 a month and breaks I-14. Technical lead.

Read as a set, these seven say one thing: this operation detects problems through people looking at screens on a schedule, not through systems watching each other. That is a defensible choice at two suppliers, one fleet and four engineers, and it stops being defensible somewhere around ten fleets or the first month where nobody has time for the Friday gate. The order in which they should be closed is O-06, O-03, O-05, O-02, O-01, O-07, O-04 — cash exposure first because it is the only one that loses money while you decide, then the measurement that would tell us whether the cost model in section 9.1 is true at all.

WHERE THIS DOCUMENT ENDS AND ANOTHER BEGINS

AA-10 carries O-01 to O-07 and the tripwires in section 9.3 into the risk register with owners and review dates. AA-08 sections 8.14 and 8.15 own the security incident path that RB-08 abbreviates. AA-01 section 1.9 owns the release protocol that section 9.6 operationalises, AA-02 section 2.7 owns the queue that section 9.5 schedules, and AA-05 owns every admin screen a runbook sends you to. If this document and one of those disagree, the other document is right about design and this one is right about what to do at 02:00.